Assignment 07: Binary Search Tree Operations

Talan Jewett · Computer Science, San Diego State University · C++17

AI Use Disclaimer. I used an AI assistant (Claude, by Anthropic) to help write the code for this assignment and to help draft this write-up. I reviewed the implementation, compiled and ran it myself, and the output shown in this document is from that run.

1. Submission Links

- **GitHub repository:** https://github.com/<your-username>/<repo-name>
- **Submitted version:** tag `assignment07-submission`
- **Build:** `g++ -std=c++17 -Wall -Wextra -pedantic -o bst main.cpp` then `./bst`
- All code is in a single file, `main.cpp`. No `std::set`, `std::map`, or other tree container is used; `std::vector` only collects traversal output.

2. Design Decisions

Node type. A plain `struct Node` holds an `int key` and raw `left`/`right` pointers defaulted to `nullptr`. Keys are distinct; inserting a duplicate returns `false` and leaves the tree unchanged.

Ownership. `BinarySearchTree` owns every node. The destructor frees the tree with a postorder walk (children before parent). Because the class manages raw memory, the copy constructor and copy assignment are deleted (rule of three) so two trees can never share and double-free the same nodes.

Pointer-to-pointer links. `insert` and `remove` walk a `Node**` that points at the link that will change (either `root_` or a parent's child pointer). This removes the special case for the root and the need to track a separate parent pointer and left/right flag.

Iterative search/insert/remove, recursive traversals. The operations that only follow one path down the tree are loops, so they use O(1) extra space. Traversals, height, and the validation check visit every node and are written recursively because that matches their definitions directly.

Height convention. Height is counted in edges: an empty tree is -1 and a single node is 0.

Removal. After locating the link to the target node:

- **Leaf:** the link is set to `nullptr` and the node is deleted.
- **One child:** the link is redirected to the only child, bypassing the node.
- **Two children:** find the inorder successor (the leftmost node of the right subtree), copy its key into the target, then splice the successor out. The successor has no left child, so splicing it out is always a leaf or one-child removal.

Leaf and one-child removal share one line of code: `*link = target->left ? target->left : target->right;` evaluates to `nullptr` for a leaf.

3. Testing Approach

`main()` is a self-checking test driver. Each check prints PASS or FAIL, and the program exits with a nonzero status if any check fails. Two test oracles are used after every change: the inorder traversal must be strictly increasing, and `isValidBST()` verifies that every node lies within the (low, high) bounds set by its ancestors, which catches errors a parent-child-only check would miss.

Test	What it shows
Empty tree	Height is -1 before any insert
Insert 11 keys: 50 30 70 20 40 60 80 35 45 65 85	Size is 11; height is 3; inorder is sorted; BST invariant holds
Duplicate insert(40)	Rejected, size unchanged
contains(45) / contains(99)	Present key found; absent key not found
remove(20)	Leaf case (0 children)
remove(60)	One-child case (only right child 65)
remove(50)	Two-child case at the root; successor 65 becomes the new root
remove(999)	Absent key returns false; size unchanged
Insert 1..10 in order	Degenerate tree with height 9 = n - 1 (worst case)

The program was also compiled with `-fsanitize=address,undefined`; it ran with no memory leaks, invalid accesses, or undefined behavior reported, and no compiler warnings under `-Wall -Wextra -pedantic` (g++ 13.3).

4. Results

All 31 checks passed. The inorder traversal stayed sorted after each of the three removals:

After	Inorder traversal
Initial inserts	`20 30 35 40 45 50 60 65 70 80 85`
remove(20) — leaf	`30 35 40 45 50 60 65 70 80 85`
remove(60) — one child	`30 35 40 45 50 65 70 80 85`
remove(50) — two children	`30 35 40 45 65 70 80 85`

Full program output is in Section 7.

5. Complexity Analysis

Let *n* be the number of nodes and *h* the tree's height. Search, insert, and remove each follow a single root-to-leaf path (remove may continue down to the successor, which is still one path), so each runs in **O(h)** time.

Operation	Average time	Worst time	Extra space
Search	O(log n)	O(n)	O(1), iterative
Insert	O(log n)	O(n)	O(1), iterative

Remove	O(log n)	O(n)	O(1), iterative
Inorder / preorder / postorder	O(n)	O(n)	O(h) recursion + O(n) output vector
Height	O(n)	O(n)	O(h) recursion

Average case. When keys arrive in random order, the expected height of a BST is O(log n), so a lookup discards roughly half of the remaining tree at each step. The 11-key demo tree has height 3, which matches floor($\log_2$ 11) = 3, the minimum possible.

Worst case. If keys arrive already sorted, every new key becomes the right child of the previous one and the tree degenerates into a linked list with height n - 1. The program demonstrates this: inserting 1 through 10 in order gives height 9, and searching for 10 visits all 10 nodes. Self-balancing trees (AVL, red-black) avoid this by keeping h = O(log n), but this assignment uses a plain BST.

Space. The tree stores n nodes, so total space is O(n). Recursion depth for traversals is O(h): O(log n) on average and O(n) for a degenerate tree. For very large degenerate trees this recursion (including in the destructor) could overflow the stack; an explicit-stack version would fix that, but it is not a concern at this assignment's size.

6. Source Code (main.cpp)

```
// Assignment 07: Binary Search Tree Operations
// Author: Talan Jewett
// Build:  g++ -std=c++17 -Wall -Wextra -pedantic -o bst main.cpp
// Run:    ./bst
//
// A binary search tree of distinct ints implemented from scratch.
// No std::set / std::map / other tree containers are used; std::vector is
// used only to collect traversal results for printing and checking.

#include <algorithm>
#include <climits>
#include <cstddef>
#include <iostream>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// Node type
// ---------------------------------------------------------------------------
struct Node {
    int key;
    Node* left = nullptr;
    Node* right = nullptr;
    explicit Node(int k) : key(k) {}
};

// ---------------------------------------------------------------------------
// BinarySearchTree
// Invariant: for every node, all keys in its left subtree are smaller and all
// keys in its right subtree are larger. Duplicates are rejected.
// ---------------------------------------------------------------------------
class BinarySearchTree {
public:
    BinarySearchTree() = default;
    ~BinarySearchTree() { destroy(root_); }

    // The tree owns raw pointers, so copying is disabled to prevent
    // double-free (rule of three).
    BinarySearchTree(const BinarySearchTree&) = delete;
    BinarySearchTree& operator=(const BinarySearchTree&) = delete;

    // Insert key. Returns false (and changes nothing) if key already exists.
    bool insert(int key) {
        Node** link = &root_;  // address of the pointer we may overwrite
        while (*link != nullptr) {
            if (key < (*link)->key)      link = &(*link)->left;
            else if (key > (*link)->key) link = &(*link)->right;
            else                         return false;  // duplicate
        }
        *link = new Node(key);
        ++size_;
        return true;
    }

    // Search for key. Returns true if present.
    bool contains(int key) const {
        const Node* cur = root_;
        while (cur != nullptr) {
            if (key == cur->key) return true;
            cur = (key < cur->key) ? cur->left : cur->right;
        }
        return false;
```

```
    }

    // Number of children of the node holding key, or -1 if key is absent.
    // Used by the demo to label which removal case is being exercised.
    int childCount(int key) const {
        const Node* cur = root_;
        while (cur != nullptr && cur->key != key)
            cur = (key < cur->key) ? cur->left : cur->right;
        if (cur == nullptr) return -1;
        return (cur->left != nullptr) + (cur->right != nullptr);
    }

    // Remove key. Returns false if key was not in the tree.
    bool remove(int key) {
        // 1. Find the link (parent's child pointer, or root_) to the target.
        Node** link = &root_;
        while (*link != nullptr && (*link)->key != key)
            link = (key < (*link)->key) ? &(*link)->left : &(*link)->right;

        Node* target = *link;
        if (target == nullptr) return false;  // not found

        if (target->left != nullptr && target->right != nullptr) {
            // Case 3: two children.
            // Find the inorder successor = minimum of the right subtree.
            Node** succLink = &target->right;
            while ((*succLink)->left != nullptr) succLink = &(*succLink)->left;
            Node* succ = *succLink;

            target->key = succ->key;  // copy successor's key into target
            *succLink = succ->right;  // splice successor out (it has no left child)
            delete succ;
        } else {
            // Case 1 (leaf) and Case 2 (one child):
            // replace the target with its only child, or nullptr for a leaf.
            *link = (target->left != nullptr) ? target->left : target->right;
            delete target;
        }
        --size_;
        return true;
    }

    std::vector<int> inorder() const   { std::vector<int> v; inorder(root_, v);   return v; }
    std::vector<int> preorder() const  { std::vector<int> v; preorder(root_, v);  return v; }
    std::vector<int> postorder() const { std::vector<int> v; postorder(root_, v); return v; }

    // Height measured in edges: empty tree = -1, single node = 0.
    int height() const { return height(root_); }

    std::size_t size() const { return size_; }
    bool empty() const { return root_ == nullptr; }

    // Verifies the BST invariant over the whole tree (used as a test oracle).
    bool isValidBST() const { return isValid(root_, LLONG_MIN, LLONG_MAX); }

    // Prints the tree rotated 90 degrees (root on the left, right subtree up).
    void printSideways(std::ostream& out) const {
        if (root_ == nullptr) { out << "    (empty)\n"; return; }
        printSideways(out, root_, 0);
    }

private:
```

```
    Node* root_ = nullptr;
    std::size_t size_ = 0;

    static void destroy(Node* n) {  // postorder delete
        if (n == nullptr) return;
        destroy(n->left);
        destroy(n->right);
        delete n;
    }

    static void inorder(const Node* n, std::vector<int>& out) {
        if (n == nullptr) return;
        inorder(n->left, out);
        out.push_back(n->key);
        inorder(n->right, out);
    }

    static void preorder(const Node* n, std::vector<int>& out) {
        if (n == nullptr) return;
        out.push_back(n->key);
        preorder(n->left, out);
        preorder(n->right, out);
    }

    static void postorder(const Node* n, std::vector<int>& out) {
        if (n == nullptr) return;
        postorder(n->left, out);
        postorder(n->right, out);
        out.push_back(n->key);
    }

    static int height(const Node* n) {
        if (n == nullptr) return -1;
        return 1 + std::max(height(n->left), height(n->right));
    }

    static bool isValid(const Node* n, long long lo, long long hi) {
        if (n == nullptr) return true;
        if (n->key <= lo || n->key >= hi) return false;
        return isValid(n->left, lo, n->key) && isValid(n->right, n->key, hi);
    }

    static void printSideways(std::ostream& out, const Node* n, int depth) {
        if (n == nullptr) return;
        printSideways(out, n->right, depth + 1);
        out << "    " << std::string(static_cast<std::size_t>(depth) * 6, ' ')
            << n->key << '\n';
        printSideways(out, n->left, depth + 1);
    }
};

// ----------------------------------------------------------------------------
// Demo / test helpers
// ----------------------------------------------------------------------------
static std::string toString(const std::vector<int>& v) {
    std::string s = "[";
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (i > 0) s += ' ';
        s += std::to_string(v[i]);
    }
    return s + "]";
}
```

```
static bool isStrictlySorted(const std::vector<int>& v) {
    for (std::size_t i = 1; i < v.size(); ++i)
        if (v[i - 1] >= v[i]) return false;
    return true;
}

static int failures = 0;

static void check(bool condition, const std::string& label) {
    std::cout << "  [" << (condition ? "PASS" : "FAIL") << "] " << label << '\n';
    if (!condition) ++failures;
}

static void printTraversals(const BinarySearchTree& t) {
    std::cout << "  Inorder:   " << toString(t.inorder()) << '\n'
              << "  Preorder:  " << toString(t.preorder()) << '\n'
              << "  Postorder: " << toString(t.postorder()) << '\n'
              << "  Height: " << t.height() << "   Size: " << t.size() << '\n';
}

static void demoRemoval(BinarySearchTree& t, int key, int expectedChildren,
                        const std::string& caseName) {
    std::cout << "\n--- Remove " << key << " (" << caseName << ") ---\n";
    int children = t.childCount(key);
    std::cout << "  Node " << key << " has " << children << " child(ren) before removal\n";
    check(children == expectedChildren, "node is the expected removal case");

    std::size_t before = t.size();
    bool removed = t.remove(key);
    check(removed, "remove(" + std::to_string(key) + ") returned true");
    check(!t.contains(key), std::to_string(key) + " is no longer found");
    check(t.size() == before - 1, "size decreased by 1");

    std::vector<int> in = t.inorder();
    std::cout << "  Inorder after removal: " << toString(in) << '\n';
    check(isStrictlySorted(in), "inorder traversal is still sorted");
    check(t.isValidBST(), "BST invariant holds for every node");
    std::cout << "  Tree now (rotated, root at left):\n";
    t.printSideways(std::cout);
}

// ----------------------------------------------------------------------------
// main
// ----------------------------------------------------------------------------
int main() {
    std::cout << "=== Assignment 07: Binary Search Tree Operations ===\n";

    BinarySearchTree tree;
    check(tree.empty() && tree.height() == -1, "empty tree has height -1");

    // 1. Insert 11 distinct integers (order chosen to give a balanced-ish tree).
    const std::vector<int> keys = {50, 30, 70, 20, 40, 60, 80, 35, 45, 65, 85};
    std::cout << "\n--- Insert " << keys.size() << " distinct keys ---\n";
    std::cout << "  Insertion order: " << toString(keys) << '\n';
    for (int k : keys) tree.insert(k);
    check(tree.size() == keys.size(), "size equals number of keys inserted");
    check(!tree.insert(40), "duplicate insert(40) is rejected");
    check(tree.size() == keys.size(), "size unchanged after duplicate");

    std::cout << "\n  Tree (rotated, root at left):\n";
    tree.printSideways(std::cout);
```

```
    std::cout << '\n';
    printTraversals(tree);
    check(isStrictlySorted(tree.inorder()), "inorder traversal is sorted");
    check(tree.isValidBST(), "BST invariant holds");
    check(tree.height() == 3, "height is 3 for this insertion order");

    // 2. Search: one present key, one absent key.
    std::cout << "\n--- Search ---\n";
    std::cout << "  contains(45) -> " << std::boolalpha << tree.contains(45) << '\n';
    check(tree.contains(45), "present key 45 is found");
    std::cout << "  contains(99) -> " << tree.contains(99) << '\n';
    check(!tree.contains(99), "absent key 99 is not found");

    // 3. Removal: all three cases.
    demoRemoval(tree, 20, 0, "Case 1: leaf");
    demoRemoval(tree, 60, 1, "Case 2: one child");
    demoRemoval(tree, 50, 2, "Case 3: two children, root; successor 65 replaces it");

    std::cout << "\n--- Remove absent key ---\n";
    std::size_t before = tree.size();
    check(!tree.remove(999), "remove(999) on absent key returns false");
    check(tree.size() == before, "size unchanged");

    std::cout << "\n--- Final state ---\n";
    printTraversals(tree);

    // 4. Worst case: sorted insertion produces a degenerate (linked-list) tree.
    std::cout << "\n--- Worst case: insert 1..10 in sorted order ---\n";
    BinarySearchTree chain;
    for (int k = 1; k <= 10; ++k) chain.insert(k);
    std::cout << "  Preorder:  " << toString(chain.preorder()) << '\n'
              << "  Height: " << chain.height() << "   Size: " << chain.size() << '\n';
    check(chain.height() == 9, "height is n - 1 = 9 (degenerate tree)");
    check(chain.contains(10), "search for 10 must visit all 10 nodes");

    std::cout << "\n=== " << (failures == 0 ? "ALL TESTS PASSED" : "SOME TESTS FAILED")
              << " (" << failures << " failure(s)) ===\n";
    return failures == 0 ? 0 : 1;
}
```

7. Program Output

```
=== Assignment 07: Binary Search Tree Operations ===
  [PASS] empty tree has height -1

--- Insert 11 distinct keys ---
  Insertion order: [50 30 70 20 40 60 80 35 45 65 85]
  [PASS] size equals number of keys inserted
  [PASS] duplicate insert(40) is rejected
  [PASS] size unchanged after duplicate

  Tree (rotated, root at left):
                        85
                80
        70
                        65
                60
    50
                        45
                40
                        35
        30
                20

  Inorder:   [20 30 35 40 45 50 60 65 70 80 85]
  Preorder:  [50 30 20 40 35 45 70 60 65 80 85]
  Postorder: [20 35 45 40 30 65 60 85 80 70 50]
  Height: 3   Size: 11
  [PASS] inorder traversal is sorted
  [PASS] BST invariant holds
  [PASS] height is 3 for this insertion order

--- Search ---
  contains(45) -> true
  [PASS] present key 45 is found
  contains(99) -> false
  [PASS] absent key 99 is not found

--- Remove 20 (Case 1: leaf) ---
  Node 20 has 0 child(ren) before removal
  [PASS] node is the expected removal case
  [PASS] remove(20) returned true
  [PASS] 20 is no longer found
  [PASS] size decreased by 1
  Inorder after removal: [30 35 40 45 50 60 65 70 80 85]
  [PASS] inorder traversal is still sorted
  [PASS] BST invariant holds for every node
  Tree now (rotated, root at left):
                        85
                80
        70
                        65
                60
    50
                        45
                40
                        35
        30

--- Remove 60 (Case 2: one child) ---
  Node 60 has 1 child(ren) before removal
  [PASS] node is the expected removal case
  [PASS] remove(60) returned true
  [PASS] 60 is no longer found
```

```
  [PASS] size decreased by 1
  Inorder after removal: [30 35 40 45 50 65 70 80 85]
  [PASS] inorder traversal is still sorted
  [PASS] BST invariant holds for every node
  Tree now (rotated, root at left):
                      85
                80
          70
                65
    50
                      45
                40
                      35
          30

--- Remove 50 (Case 3: two children, root; successor 65 replaces it) ---
  Node 50 has 2 child(ren) before removal
  [PASS] node is the expected removal case
  [PASS] remove(50) returned true
  [PASS] 50 is no longer found
  [PASS] size decreased by 1
  Inorder after removal: [30 35 40 45 65 70 80 85]
  [PASS] inorder traversal is still sorted
  [PASS] BST invariant holds for every node
  Tree now (rotated, root at left):
                      85
                80
          70
    65
                      45
                40
                      35
          30

--- Remove absent key ---
  [PASS] remove(999) on absent key returns false
  [PASS] size unchanged

--- Final state ---
  Inorder:   [30 35 40 45 65 70 80 85]
  Preorder:  [65 30 40 35 45 70 80 85]
  Postorder: [35 45 40 30 85 80 70 65]
  Height: 3   Size: 8

--- Worst case: insert 1..10 in sorted order ---
  Preorder:  [1 2 3 4 5 6 7 8 9 10]
  Height: 9   Size: 10
  [PASS] height is n - 1 = 9 (degenerate tree)
  [PASS] search for 10 must visit all 10 nodes

=== ALL TESTS PASSED (0 failure(s)) ===
```